

Active Directory Password Reset Credential Provider

Project Name: AD Password Reset Credential Provider
Prepared By: IT Department – VizPay
Date: 23 April 2026
Version: 1.0

1. Executive Summary

This project delivers a self-service **Active Directory (AD) password reset solution** that appears directly on the **Windows lock screen** of domain-joined laptops. Users who have forgotten their password can reset it without calling the IT helpdesk, by verifying their identity through a One-Time Password (OTP) sent to their registered company email address.

2. Problem Statement

Current Situation

- When a domain user forgets their Windows login password, they must call the IT helpdesk.
- The helpdesk manually resets the password in Active Directory and communicates it to the user.
- This wastes IT staff time, causes delays, and creates a security risk (passwords communicated verbally or via unencrypted channels).

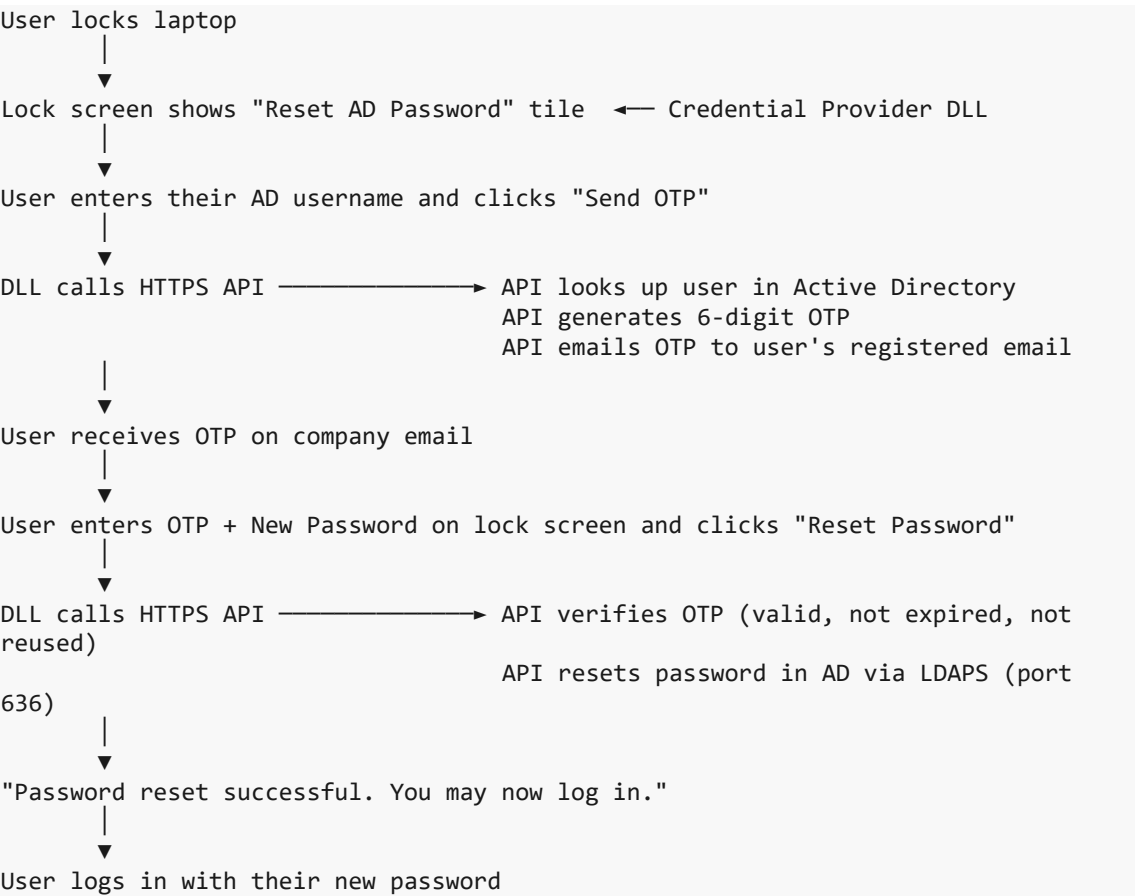
Goal

Allow users to reset their own Active Directory password securely from the lock screen of their laptop, without any IT helpdesk involvement.

3. Solution Overview

Component	Technology	Runs On
Windows Credential Provider DLL	C++ (Win32 COM)	User's domain-joined laptop
Password Reset API (Backend)	ASP.NET Core 8 (.NET)	Domain Controller / Internal Server

End-to-End Flow



4. Component 1 – Windows Credential Provider DLL

A Windows Credential Provider is a COM (Component Object Model) DLL that Microsoft allows third parties to register on Windows. Once registered, it appears as a tile on the Windows lock/logon screen alongside the standard tile.

Item	Detail
Language	C++ (C++17)
APIs Used	Windows Credential Provider API, WinHTTP, COM
Build Tool	MSBuild — Visual Studio 2025 (toolset v145)
Output File	ResetPasswordCP.dll (64-bit)

Source Files

File	Purpose
src\dllmain.cpp	DLL entry point, COM factory, registry registration
src\CredentialProvider.cpp	ICredentialProvider — manages tile and credential object
src\Credential.cpp	ICredentialProviderCredential — UI fields and user interaction
src\HttpClient.h	WinHTTP wrapper — HTTPS POST to backend API
ResetPasswordCP.def	DLL export definitions
ResetPasswordCP.vcxproj	Visual Studio project (MSBuild)

UI Fields on Lock Screen

Field	Type	Purpose
Reset AD Password	Header text	Tile title — visible when collapsed
Username	Text input	User enters their AD username
Send OTP	Command link	Triggers OTP generation and email
OTP Code	Password field (masked)	User enters OTP from email
New Password	Password field (masked)	User enters desired new password
Status	Status text	Live feedback — errors and success messages

Reset Password	Submit button	Triggers password reset via API
----------------	---------------	---------------------------------

5. Component 2 – Password Reset API (Backend)

Item	Detail
Framework	ASP.NET Core 8 Minimal API
Language	C# (.NET 8)
Deployment	Windows Service — auto-starts on boot
HTTPS	Kestrel with PFX certificate on port 8443
AD Integration	System.DirectoryServices.Protocols — LDAPS port 636
Email	Zoho SMTP (smtp.zoho.com:587) with SSL
OTP Storage	In-memory (no database required)
Rate Limiting	Max 10 requests per 15-minute window per IP

API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/request-otp	Validates username in AD, generates OTP, sends email
POST	/api/auth/reset-password	Verifies OTP, resets AD password via LDAPS
GET	/health	Health check — returns 200 OK if running

6. Infrastructure & Network

Item	Detail
API Server IP	172.16.10.4 (Domain Controller)
API Hostname	reset.viz.in
API Port	8443 (HTTPS only — no plain HTTP)
AD Domain	viz.in
LDAPS Port	636 (encrypted AD communication)
SMTP Provider	Zoho Mail (smtp.zoho.com:587 with SSL)
TLS Certificate	Internal certificate issued for CN=reset.viz.in
Firewall Rule	Inbound TCP 8443 allowed on DC — all other ports blocked

7. Why This Solution Is Secure

Every part of this system was designed with security as the top priority. Below is a detailed explanation of each protection layer built into the solution.

7.1 Identity Verification via OTP (Email-Based)

What it does: Before any password reset is allowed, the system sends a 6-digit One-Time Password to the email address stored in Active Directory for that user account.

Why it is secure: Only the legitimate owner of the account has access to the registered company email inbox. An attacker who knows someone's username cannot reset their password without also accessing their email. This is two-factor verification: something you know (username) + something you have (access to email).

7.2 OTP Expires in 5 Minutes

What it does: Each OTP is valid for only 300 seconds (5 minutes). After expiry it is automatically deleted and cannot be used.

Why it is secure: Even if an OTP is somehow intercepted or leaked, it becomes useless within minutes. This drastically reduces the window of opportunity for any attack.

7.3 OTP is Single-Use Only

What it does: Once an OTP is used (whether the reset succeeds or fails), it is immediately deleted from memory and cannot be used again.

Why it is secure: Prevents replay attacks — an attacker who captures an OTP cannot reuse it. Even if a user accidentally shares their OTP, it is already invalid after first use.

7.4 Rate Limiting — Brute Force Protection

What it does: The API allows a maximum of 10 OTP requests per IP address per 15-minute window. Any request beyond this limit is rejected (HTTP 429 Too Many Requests).

Why it is secure: An attacker cannot try thousands of OTP combinations in a short time. A real user only needs 1–2 OTP requests, so the limit never affects legitimate users.

7.5 All Communication Encrypted with HTTPS (TLS)

What it does: Every request between the laptop DLL and the backend API travels over HTTPS on port 8443. Plain HTTP is not supported — not even as a fallback.

Why it is secure: Network traffic cannot be read by anyone monitoring the network. Usernames, OTPs, and new passwords are never visible on the wire. This is the same encryption standard used by online banking.

7.6 AD Password Change over LDAPS (Port 636)

What it does: The API connects to Active Directory using LDAPS (LDAP over SSL) on port 636 to change the user's password.

Why it is secure: Microsoft requires an encrypted channel to change AD passwords — it is impossible to change a password over plain LDAP on port 389. The new password is never transmitted in plain text, even on the internal network.

7.7 Dedicated Minimum-Privilege Service Account

What it does: The API connects to AD using a dedicated service account with only one permission: **Reset Password on user objects**. It cannot read other AD attributes, create or delete accounts, or change group memberships.

Why it is secure: Follows the principle of least privilege. If the service account credentials were ever compromised, the attacker gains only the ability to reset passwords — and that still requires a valid OTP sent to the user's email.

7.8 Passwords Wiped from Memory Immediately

What it does: In the DLL, as soon as the new password is sent to the API, `SecureZeroMemory()` is called to overwrite the password buffer with zeros before releasing it.

Why it is secure: `SecureZeroMemory()` is a Windows API specifically designed to prevent the compiler from optimizing away the memory wipe. After the call, the password no longer exists anywhere in the DLL's memory — it cannot be recovered from a process dump.

7.9 API Only Accessible Inside the Internal Network

What it does: Port 8443 is only open on the internal network. The API is not published to the internet.

Why it is secure: External attackers on the internet cannot reach the API at all. Only devices connected to the VizPay internal network or VPN can make requests. This eliminates all internet-based attack vectors.

7.10 No Password Stored Anywhere in the System

What it does: The system never stores the user's new password in any database, log file, or long-term memory.

Why it is secure: There is no password database to breach. Logs only record events (OTP requested, reset succeeded) — never the actual password or OTP value. Nothing sensitive is persisted.

7.11 Runs as a Windows Service

What it does: The backend API runs as a Windows Service that starts automatically when the server boots, independent of any user logging in.

Why it is secure: No user needs to remain logged in to the server for the service to operate. Windows Service accounts are tightly controlled by the OS and cannot be easily tampered with by unprivileged users.

Security Summary

Threat	Protection
Attacker knows username and tries to reset password	OTP sent only to registered email — requires email access
OTP intercepted in transit	HTTPS encryption — OTP never visible on the network
Attacker tries all possible OTPs	Rate limiting — max 10 attempts per 15 minutes
OTP captured and reused	Single-use OTP, expires in 5 minutes
New password intercepted in transit	HTTPS (TLS) encryption end-to-end
Password visible in memory after reset	SecureZeroMemory() wipes it immediately
Service account credentials compromised	Minimum privilege — Reset Password only, OTP still required
API attacked from the internet	Port 8443 blocked at perimeter — internal only
Password database breached	No password storage anywhere in the system
LDAP traffic sniffed on internal network	LDAPS (port 636) encrypts all AD communication

8. Deployment Steps

Backend API — Domain Controller

1. Publish .NET app as a self-contained single-file EXE (no .NET runtime needed on server)

2. Copy to C:\ResetPasswordAPI\
3. Place TLS certificate PFX at C:\ResetPasswordAPI\certs\reset-api.pfx
4. Create .env configuration file with AD and SMTP credentials
5. Open port 8443 in Windows Firewall
6. Install and start as Windows Service:
7.

```
sc create ResetPasswordAPI binPath=
"C:\ResetPasswordAPI\ResetPasswordAPI.exe" start= auto
sc start ResetPasswordAPI
```

DLL — Each User Laptop

1. Copy ResetPasswordCP.dll to C:\Windows\System32\
2. Register with Windows (elevated PowerShell):

```
regsvr32 /s C:\Windows\System32\ResetPasswordCP.dll
```
3. Lock the screen (Win+L) — tile appears immediately, no restart needed

Uninstall

```
regsvr32 /u /s C:\Windows\System32\ResetPasswordCP.dll
del C:\Windows\System32\ResetPasswordCP.dll
```

9. Testing Results

Test Case	Result
OTP sent to registered email	PASS
OTP verified and password reset in AD	PASS
Lock screen tile appears on domain laptop	PASS
Invalid username returns error message	PASS
Expired OTP rejected	PASS
API health check returns 200 OK	PASS
API auto-starts after server reboot	PASS

Rate limiting blocks excess requests	PASS
--------------------------------------	------

10. Known Limitations & Future Improvements

Item	Detail
Email requirement	User must have an email address set in their AD account (mail attribute)
OTP in memory	If the API service restarts, pending OTPs are lost — user simply requests a new one
Single server	Currently runs on the DC — future: move to a dedicated member server
Bulk laptop deployment	Currently manual per laptop — can be automated via Group Policy (GPO) or Intune
Audit logging	Future: structured logs to SIEM or database for compliance and audit trail

11. Project File Structure

D:\reset password\	
└─ src\	
└─ dllmain.cpp	DLL entry, COM exports, registry registration
└─ CProvider.h	Credential provider class declaration
└─ CredentialProvider.cpp	ICredentialProvider implementation
└─ Credential.h	Credential class + field layout
└─ Credential.cpp	ICredentialProviderCredential + UI logic
└─ HttpClient.h	WinHTTP HTTPS client
└─ ResetPasswordCP.def	DLL export definitions
└─ ResetPasswordCP.vcxproj	Visual Studio project file
└─ x64\Release\	
└─ ResetPasswordCP.dll	Final DLL – deploy this to laptops
└─ backend-dotnet\	
└─ Program.cs	Complete .NET 8 API source
└─ ResetPasswordAPI.csproj	
└─ .env.example	Configuration template
└─ install-service.ps1	Windows Service install script